

CS.105 Programming Foundations

Practical 12 - to be completed preferably by the end of your lab in week 4 (S2)

- The aim of this practical is to develop your skills in writing tests.
- The third class-test for CS105 (semester 2 week 6) will include tasks involving testing and debugging (although not use of a debugger) and so it is important to undertake this lab exercise in preparation for that class test.
- Some feedback will be available on MyPlace after you have attempted this practical.
- Demonstrators will be available in the labs to provide help on request during the CS105 lab times – please make good use of them if you are struggling or have questions.

In this practical you will try out some testing using BlueJ but are then encouraged to try out similar activities using the same projects in IntelliJ.

Preliminary work:

In BlueJ, open the online-shop project from the Chapter 9 projects. You will find it helpful to briefly explore and gain some familiarity with using the `online-shop` project – create an instance of a `SalesItem` and try invoking methods available to you. This will not take long and will help you complete the exercises on testing which follow below.

Part A Testing Basics (Don't use JUnit testing in part A of the practical.)

1. In BlueJ, within the online-shop project, create a `SalesItem` object. Open an inspector on it and another on its comments field.
2. Add several comments to the sales item while keeping an eye on the inspector for the comments list. Make sure the comments list behaves as expected (in that its size should increase). You may also like to inspect its `elementData` field and drill in to entries there.
3. Check that the `showInfo` method correctly prints the sales item information, including the comments.
4. Create a further sales item without any comments and try out `showInfo` for it also.
5. Check that the `getNumberOfComments` method works as expected.
6. Now check that an attempt by an author to enter a 2nd comment for the same sales item is not successful. When trying to add a comment with an author name for whom a comment already exists, the `addComment` method should return false. Check also that the comment was not added to the list of comments.
7. Start performing boundary value checking on the rating value. Here, create comments with ratings at the top and bottom of the permitted range (1-5). Does this work correctly?
8. Good boundary value testing involves testing values on each boundary and on either side of the boundaries. Perform tests with ratings just beyond the valid range of data, i.e. with ratings 0 and 6. It is also a good idea to perform tests on the boundaries (ratings 1 and 5) and just inside the boundaries (ratings 2 and 4). Do all these tests produce correct results?

9. Test the upVoteComment and downVoteComment methods. Make sure that the vote balance is correctly counted.
10. Use the upVoteComment and downVoteComment methods to mark some comments as more or less helpful. Then test the findMostHelpfulComment method. This method should return the comment that was voted most helpful. You will notice that the method returns an object reference. You can use the Inspect function in the method result dialogue to check whether the correct comment was returned. Of course, you will need to know which is the correct comment in order to check whether you get the right result!
11. Try findMostHelpfulComment on a sales item which has no comments. Do you think the resulting action is acceptable?

The above are just a selection of tests that could be appropriate in such a project. Testing in the above way is laborious and tests should be rerun after any code corrections / changes are made – regression testing.

Part B Automated Testing with JUnit

Switch to using the Chapter9 online-shop-junit project in BlueJ. From now on in this practical you should use JUnit testing. This greatly assists with regression testing compared to the approach in Part A.

1. Run the tests already available in this project, using the Run Tests button. All 3 tests should show as passed with a green tick. Close the test results window.
2. By double-clicking on the green test class box for SalesItem, take a look at the code within the test methods of the test class, paying particular attention to the lines containing assertions. (This code was generated automatically rather than written by hand – although it could have been entered by hand.)

Now add some further JUnit tests of your own to the test class for the exercises below. To do this in each case, right click on the test class and select Create Test Method. Give your test an appropriate name in each case below and then recording of your subsequent actions will take place, e.g. creating a SalesItem object, invoking appropriate methods on it and asserting what the results should be for methods which return results. Stop recording when you have performed this in each case. To stop recording choose Testing from the Tools menu and select End Recording. Then run the tests.

Note: you can also edit the test class code directly rather than work with recordings of tests.

3. Create a JUnit test(s) that checks that a comment with author “A. Complainer” can be stored with rating 1 for a sales item. Run the test to ensure the expected result is produced.
4. Create a JUnit test to check that addComment returns false when a comment from the same author already exists for a sales item. The assertion should state the expected result, not the actual result. Run all tests again, or just this test.
5. Create a JUnit test (or tests) that performs **boundary value testing** of the rating range. That is, test the values on and at either side of 1 and 5 as a rating (the values on, just outside and just inside the legal range). Run the test(s). If you have written the tests correctly you will notice that one of these tests fails. (You should also have found this problem earlier during manual testing in Part A.)

Run all tests again. Explore how the Test Result dialogue displays the failed test. Select the failed test in the list and explore options available to you to explore the reason for failure.

Correct the erroneous code which resulted in the failed test. Run all tests to receive a green tick/bar rather than a red tick/bar for this test.

6. In Part A exercise 11 above you may have found a problem which caused an error to occur which terminated the program running. Try to fix this error, so that method `findMostHelpfulComment` returns null if there are no comments. Then write a JUnit test to check this desired result when the `findMostHelpfulComment` method is invoked on an item where no comments have been made.
7. Create a JUnit test class that has `Comment` as its reference class. Create a test that checks whether the author and rating details are stored correctly after creation. Record separate tests that check whether upvote and downvote methods work as expected.
8. Open the bricks project from the Chapter9 projects. There are at least four errors in this project. Do some testing (although you need not produce JUnit tests in this question unless you wish to). See if you can find the errors in the code and fix them. You may choose to identify the errors by looking at the code visually or you may choose to use a debugger if you find it helpful – notes and demo of debuggers appear later in week 4 on MyPlace.

Extra Optional Challenge:

Create JUnit tests for `SalesItem` that test whether the `findMostHelpfulComment` method works as expected, identifying the correct comment. Note that this method returns a `Comment` object. During your testing (and recording of a test), you can use the Get button in the method result dialog to get the result object onto the object bench, which then allows you to make further method calls and add assertions for this object. This allows you to identify the comment object returned (e.g. by checking its author). You can also assert that the result is *null* or *not null*, depending on what you expect.

The exercises above make use of BlueJ. You are now encouraged to explore carrying out similar activities using IntelliJ.